Bootcamp - Git(Hub) : versionner & collaborer (1/2)

Xavier Clerc - xavier.clerc@enpc.fr
6 novembre 2026

Plan

- Introduction

- Gestion de versions : Git

- Manipulations

Introduction

Équipe

- Colas Drutinus-Cabin (EDF)
- Julien Cardinal (Inria)
- Liza Belos (CNRS)
- Matéo Vallouis (indépendant)
- Nicolas Bazin (GAEL Systems)
- Pascal Monasse (Imagine, ENPC)
- Rachel Roux (Airbus)
- Xavier Clerc (Jane Street)

Objectifs

- Opérations élémentaires de Git

- Utilisation de GitHub

- Interface VS Code

Objectifs

- Outils & concepts utiles...
- ... pour le développement
- ... pour la productivité
- ... pour l'ingénierie
- ... pour l'encadrement d'agents IA

Organisation

- Transparents puis manipulation

- Git puis GitHub

- (Utilisation de GitHub Codespaces / pas d'installation)

Gestion de versions : Git

Gestion de versions

- Garder la trace des évolutions des fichiers
- Permettre de consulter les versions passées
- Faciliter les expérimentations
- Simplifier la collaboration
- *Pas spécifique au développement logiciel*

Gestion de versions

- Sans outil dédié, on maintient de multiples fichiers / répertoires

- `-v1`, `-final`, `-2026-10-11`, *etc*

- Difficile de comparer les versions

- Difficile de comprendre les évolutions

diff & patch

first.py

```
def say(name: str, msg: str) -> None:
    print(f"[{name}] {msg}")

LOGINS : list[str] = []

def start_chat(login: str) -> None:
    say(login, "hello!")
```

diff

second.py

```
def say(name: str, msg: str) -> None:
    print(f"{name}: {msg}")

def start_chat(login: str) -> None:
    say(login, "hello!")

def end_chat(login: str) -> None:
    say(login, "bye!")
```

```
@@ -1,7 +1,8 @@
 def say(name: str, msg: str) -> None:
-    print(f"[{name}] {msg}")
+    print(f"{name}: {msg}")

-LOGINS : list[str] = []
-
 def start_chat(login: str) -> None:
     say(login, "hello!")
+
+def end_chat(login: str) -> None:
+    say(login, "bye!")
```

diff & patch

first.py

```
def say(name: str, msg: str) -> None:
    print(f"[{name}] {msg}")

LOGINS : list[str] = []

def start_chat(login: str) -> None:
    say(login, "hello!")
```

diff

second.py

```
def say(name: str, msg: str) -> None:
    print(f"{name}: {msg}")

def start_chat(login: str) -> None:
    say(login, "hello!")

def end_chat(login: str) -> None:
    say(login, "bye!")
```

```
@@ -1,7 +1,8 @@
 def say(name: str, msg: str) -> None:
-    print(f"[{name}] {msg}")
+    print(f"{name}: {msg}")

-LOGINS : list[str] = []
-
 def start_chat(login: str) -> None:
     say(login, "hello!")
+
+def end_chat(login: str) -> None:
+    say(login, "bye!")
```

diff & patch

first.py

```
def say(name: str, msg: str) -> None:
    print(f"[{name}] {msg}")

LOGINS : list[str] = []

def start_chat(login: str) -> None:
    say(login, "hello!")
```

patch

```
@@ -1,7 +1,8 @@
 def say(name: str, msg: str) -> None:
-    print(f"[{name}] {msg}")
+    print(f"{name}: {msg}")

-LOGINS : list[str] = []
-
 def start_chat(login: str) -> None:
     say(login, "hello!")
+
+def end_chat(login: str) -> None:
+    say(login, "bye!")
```

```
def say(name: str, msg: str) -> None:
    print(f"{name}: {msg}")

def start_chat(login: str) -> None:
    say(login, "hello!")

def end_chat(login: str) -> None:
    say(login, "bye!")
```

Gestion de versions

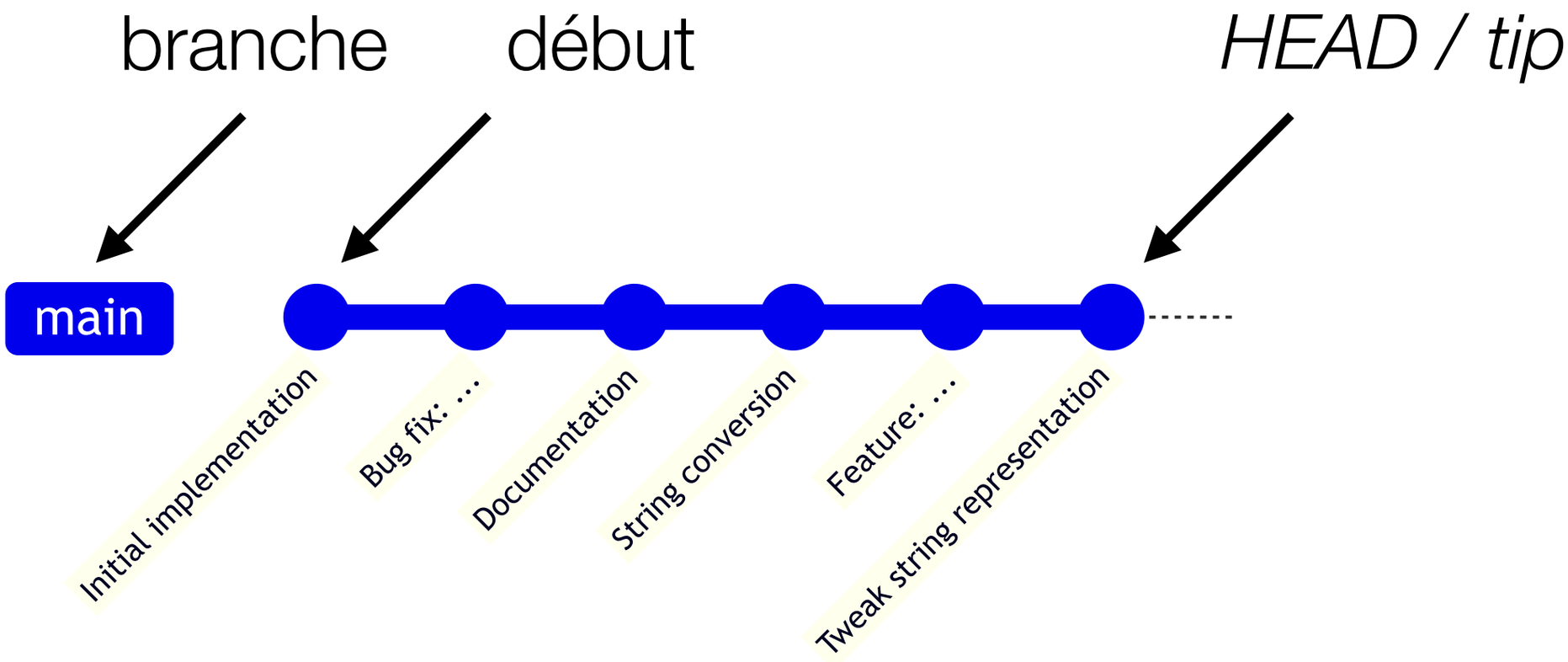
- Organiser un ensemble de patches...

- ... sous la forme d'un historique

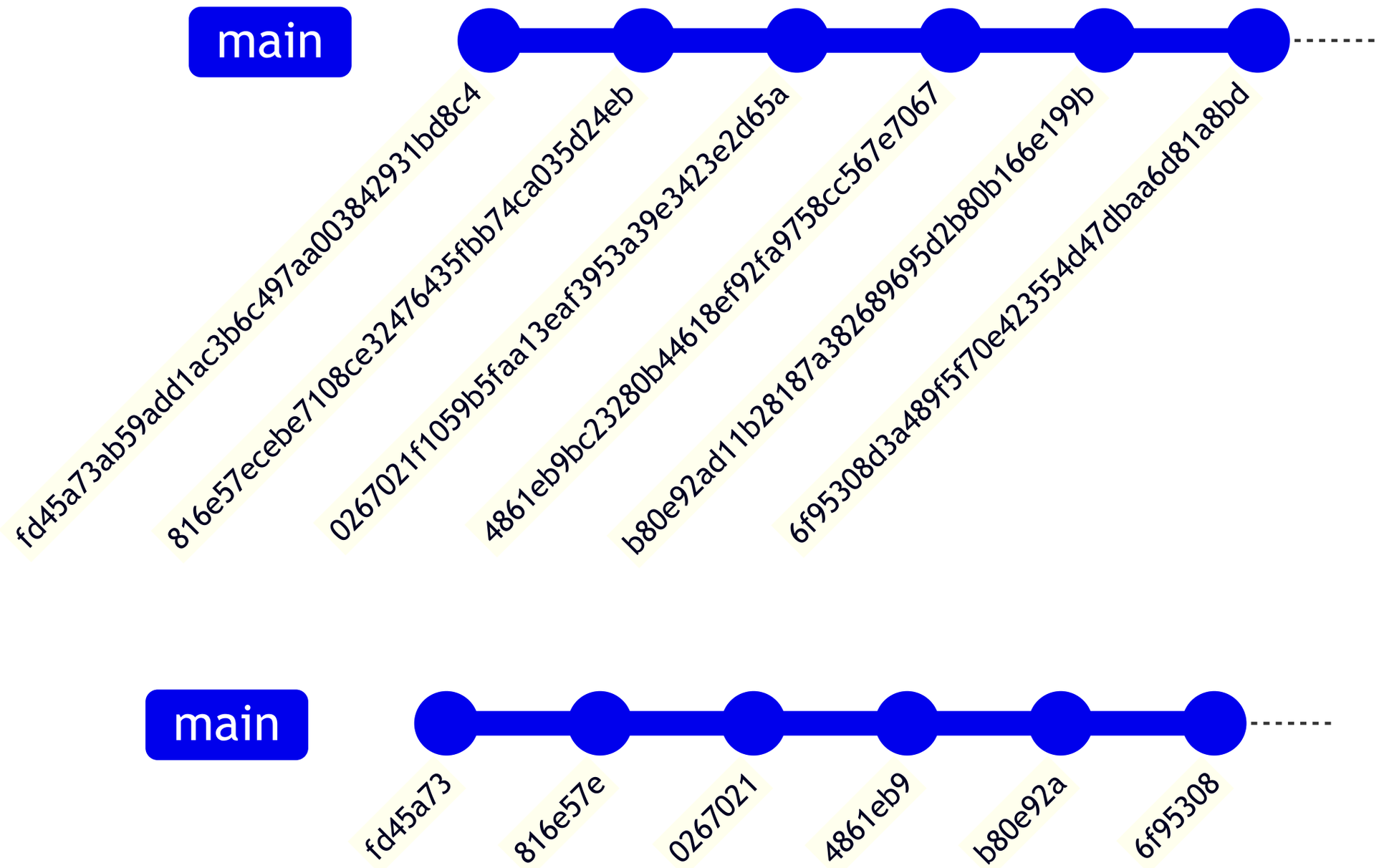
- ... avec des *metadonnées*

- ... (date, auteur, description, *etc*)

Historique


branche
début
HEAD / tip
main
Initial implementation
Bug fix: …
Documentation
String conversion
Feature: …
Tweak string representation

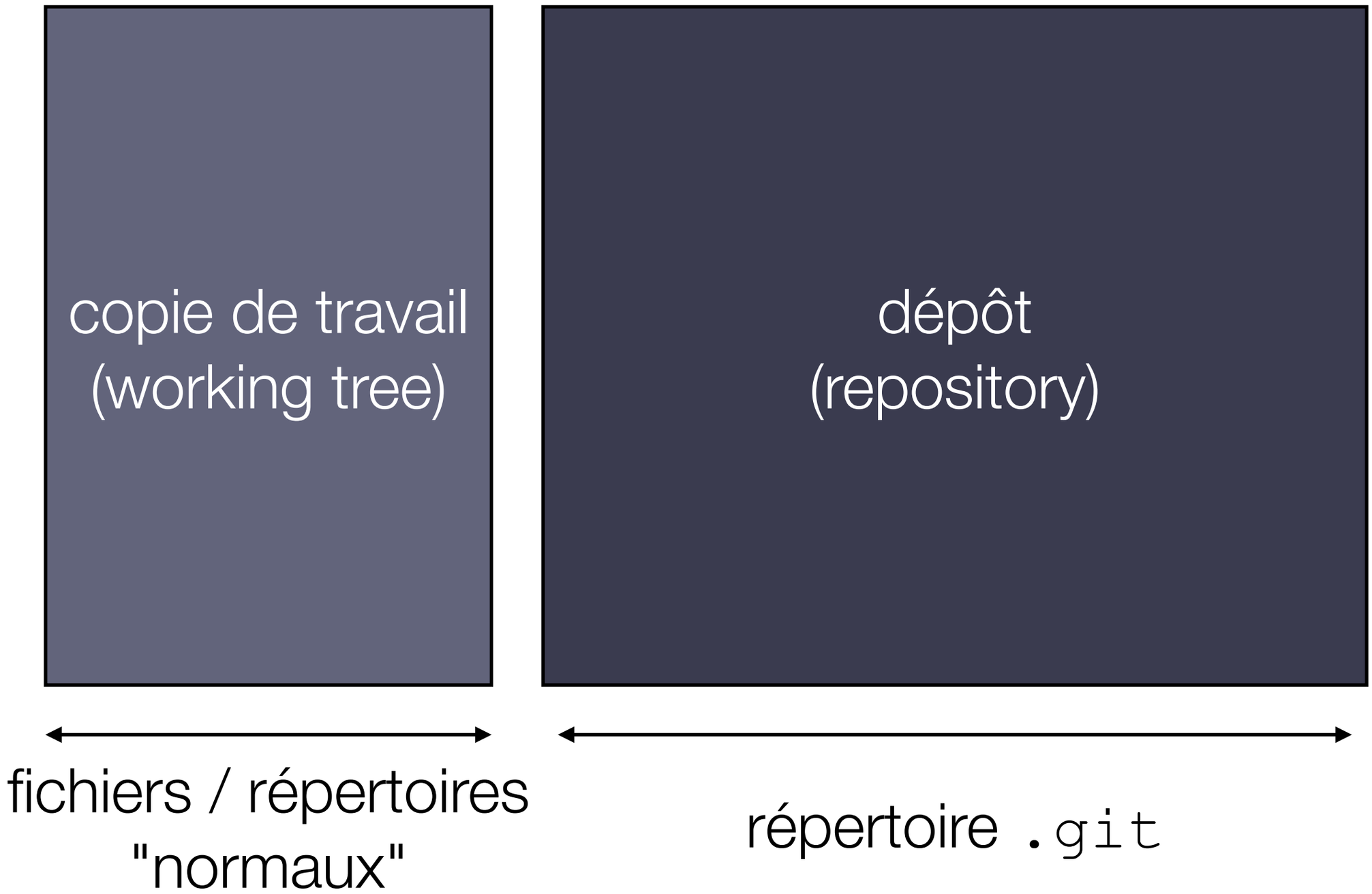
Historique


main
fd45a73ab59add1ac3b6c497aa00842931bd8c4
816e57ecebe7108ce32476435fbb74ca035d24eb
0267021f1059b5faa13eaf3953a39e3423e2d65a
4861eb9bc23280b44618ef92fa9758cc567e7067
b80e92ad11b28187a38268695d2b80b166e199b
6f95308d3a489f5f70e42554d47dbaa6d81a8bd
main
fd45a73
816e57e
0267021
4861eb9
b80e92a
6f95308

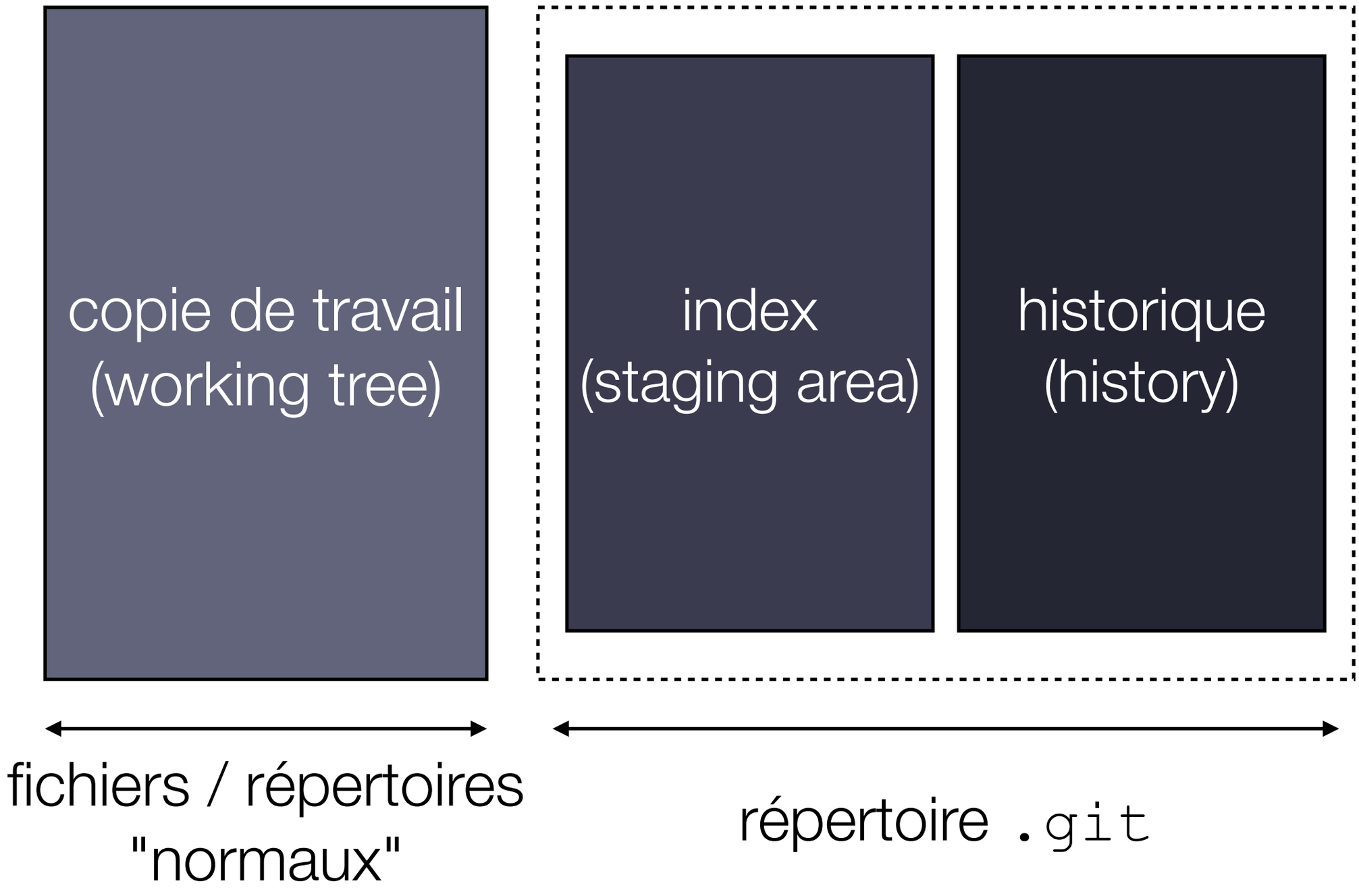
Git

- Un gestionnaire de versions populaire
- Ligne de commande et interfaces graphiques
- On se focalise ici sur les concepts de base
- On passe en revue les opérations élémentaires
- On mentionne quelques bonnes pratiques

Zones


copie de travail
(working tree)
dépôt
(repository)
fichiers / répertoires
"normaux"
répertoire .git

Zones


copie de travail
(working tree)
index
(staging area)
historique
(history)
fichiers / répertoires
"normaux"
répertoire .git

Création ou clonage

- (pour référence, préconfiguré pour cette séance)

- `git init`

- `git clone ...`

Configuration (1/2)

- (pour référence, préconfiguré pour cette séance)
- `git config --add user.name '...'`
- `git config --add user.email '...'`
- `git config --add core.editor nano`
- `--global` pour appliquer à tous les dépôts

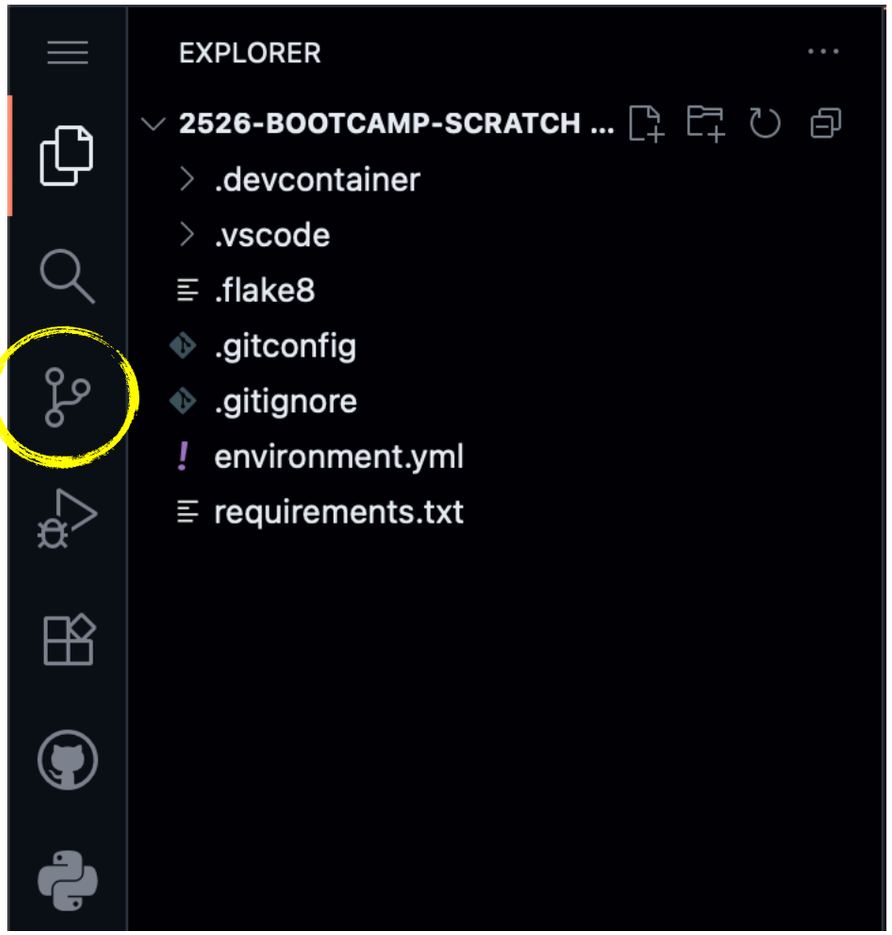
Configuration (2/2)

- (pour référence, préconfiguré pour cette séance)

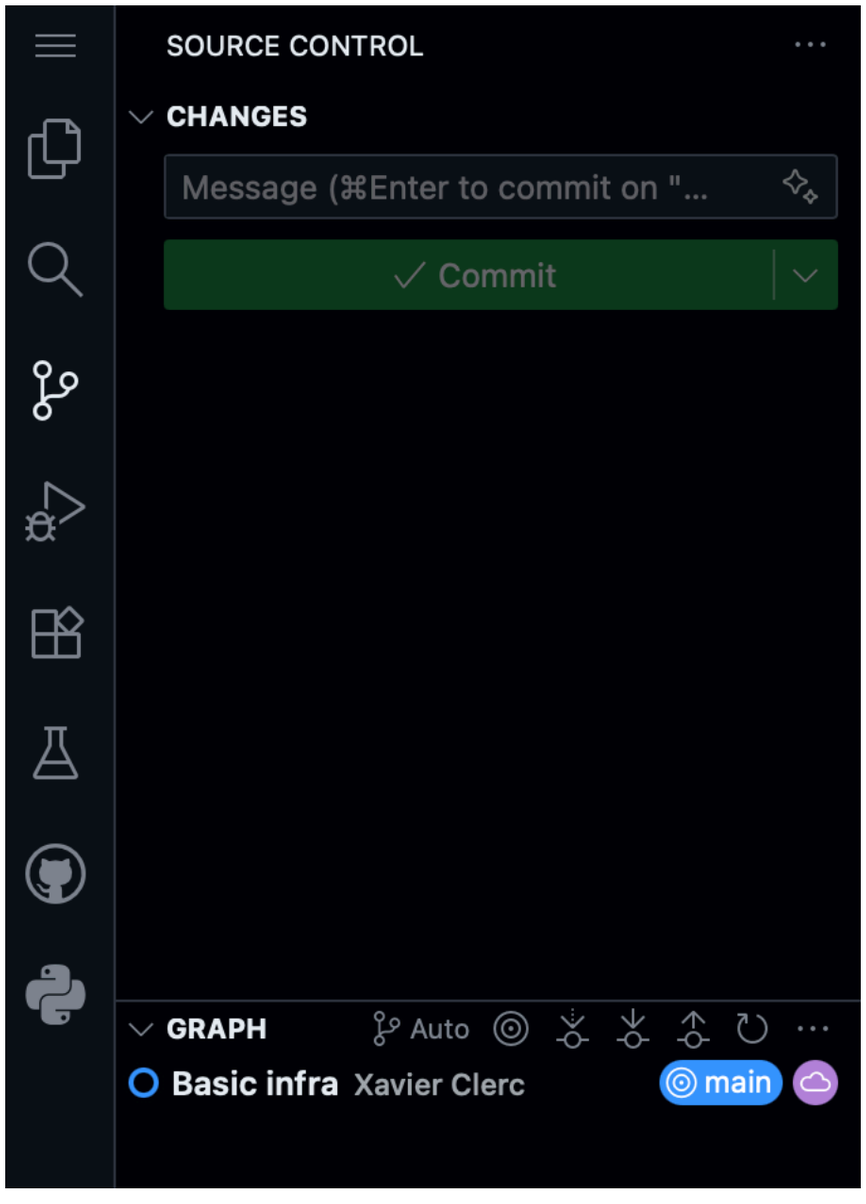
- `https://docs.github.com/en/get-started/onboarding/getting-started-with-your-github-account`

- `https://docs.github.com/en/authentication/connecting-to-github-with-ssh/adding-a-new-ssh-key-to-your-github-account`

Panneau de gestion de versions



Panneau de gestion de versions



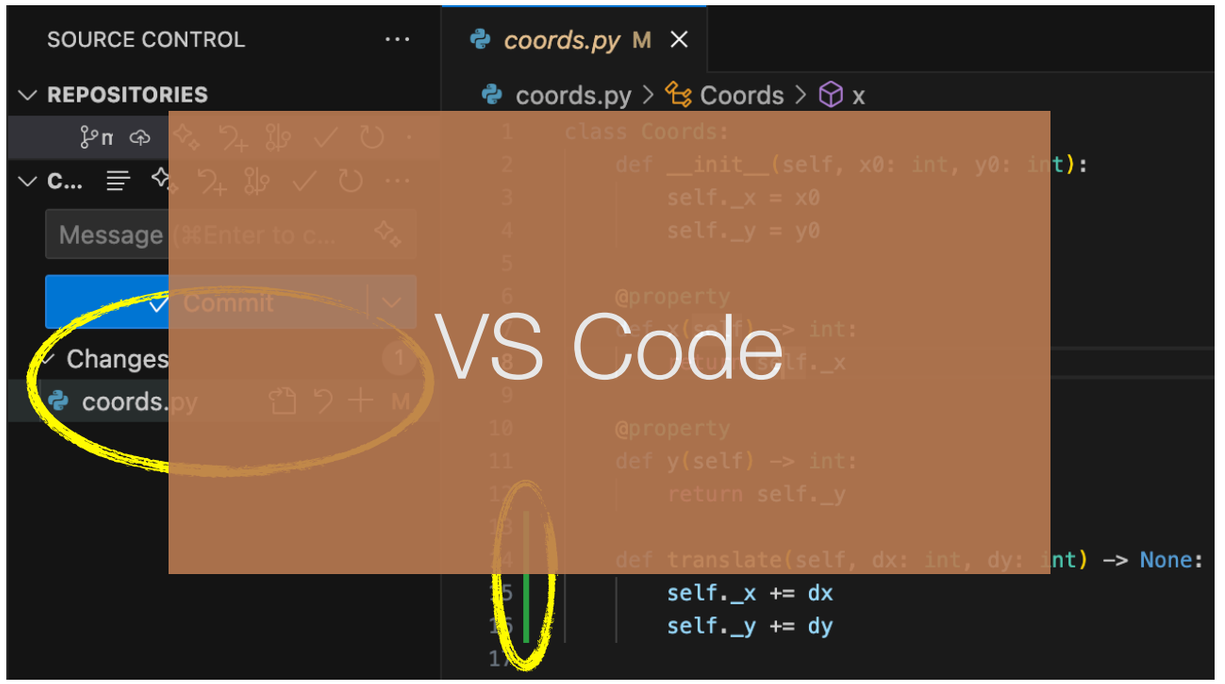
Légende






```
--- a/coords.py
+++ b/coords.py
@@ -10,3 +10,7 @@ class Coords:
     @property
     def y(self) -> int:
         return self._y
+
+    def translate(self, dx: int, dy: int) -> None:
+        self._x += dx
+        self._y += dy
```

Terminal



VS Code

Modifications non enregistrées

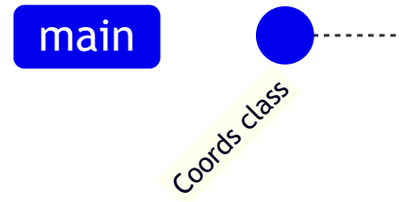
- `git diff`

- Consultation des modifications depuis le dernier commit

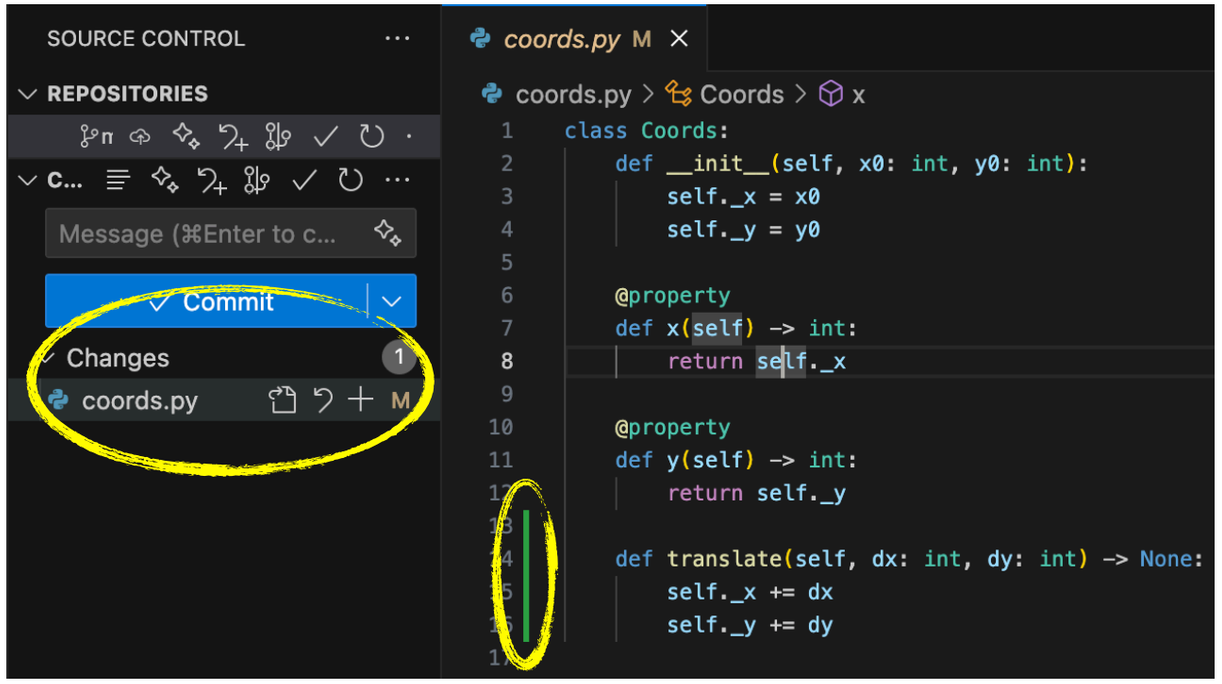
- (Pour les fichiers *suivis*)

`git diff`






```
--- a/coords.py
+++ b/coords.py
@@ -10,3 +10,7 @@ class Coords:
     @property
     def y(self) -> int:
         return self._y
+
+    def translate(self, dx: int, dy:
int) -> None:
+        self._x += dx
+        self._y += dy
```



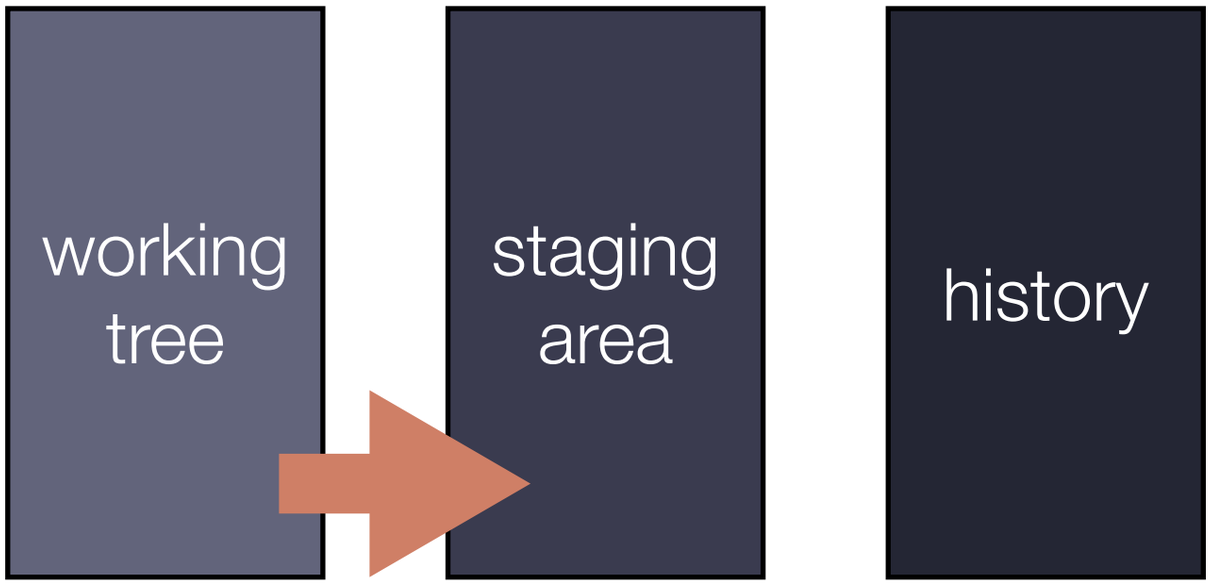
Ajout de modifications

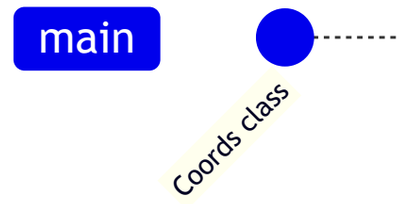
- `git add <files>`

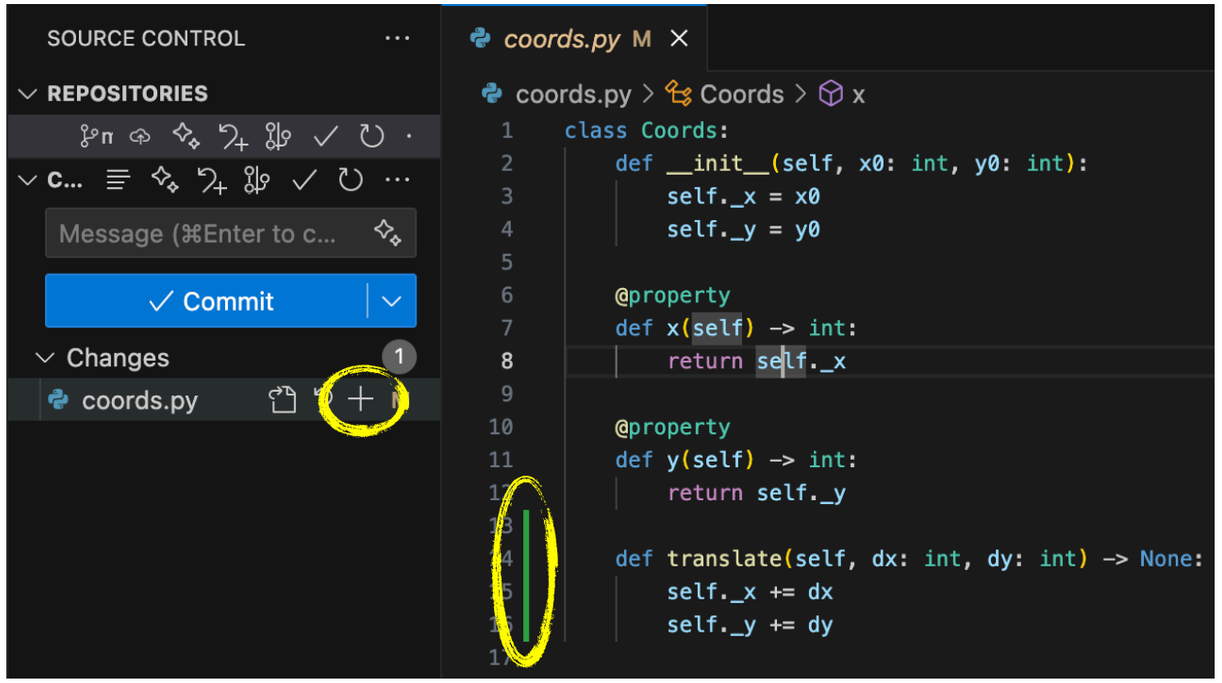
- Ajout des modifications depuis le dernier commit

- (Fichier entier si pas déjà suivi)

`git add` *files*







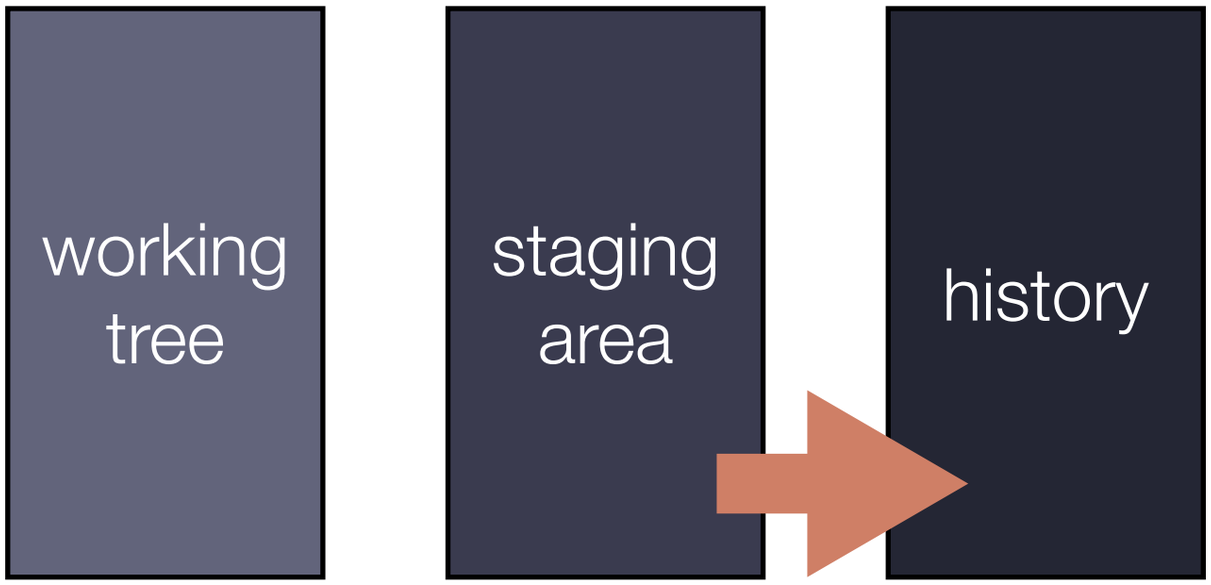
Enregistrement des modifications

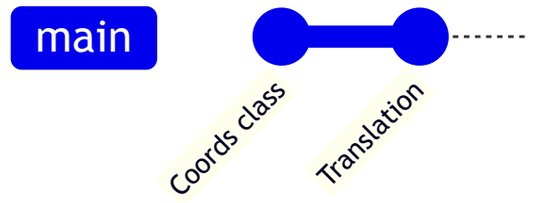
- `git commit -m '...'`

- Enregistrement des modifications avec message

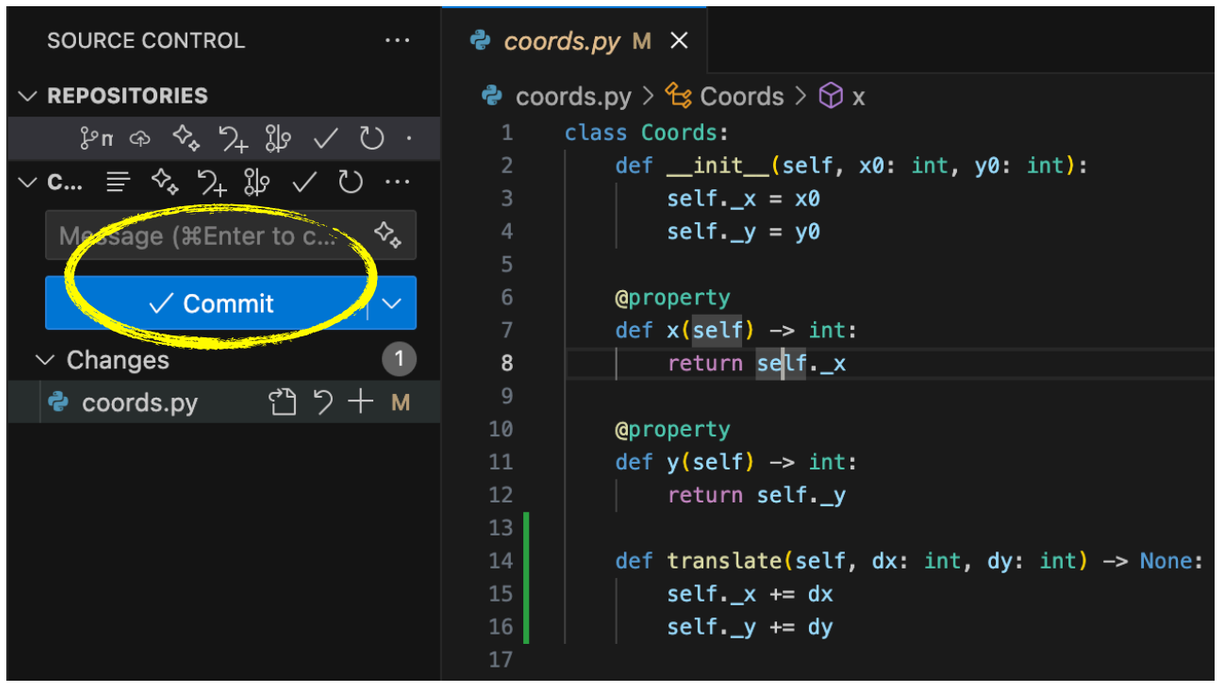
- (Identifiant et métadonnées automatiquement ajoutés)

`git commit -m "message"`






```
[main 816e57e] Translation
 1 file changed, 4 insertions(+)
```



État

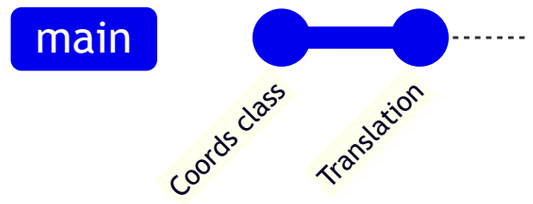
- `git status`

- Liste des fichiers avec des changements

- (et des fichiers non-suivis ou supprimés)

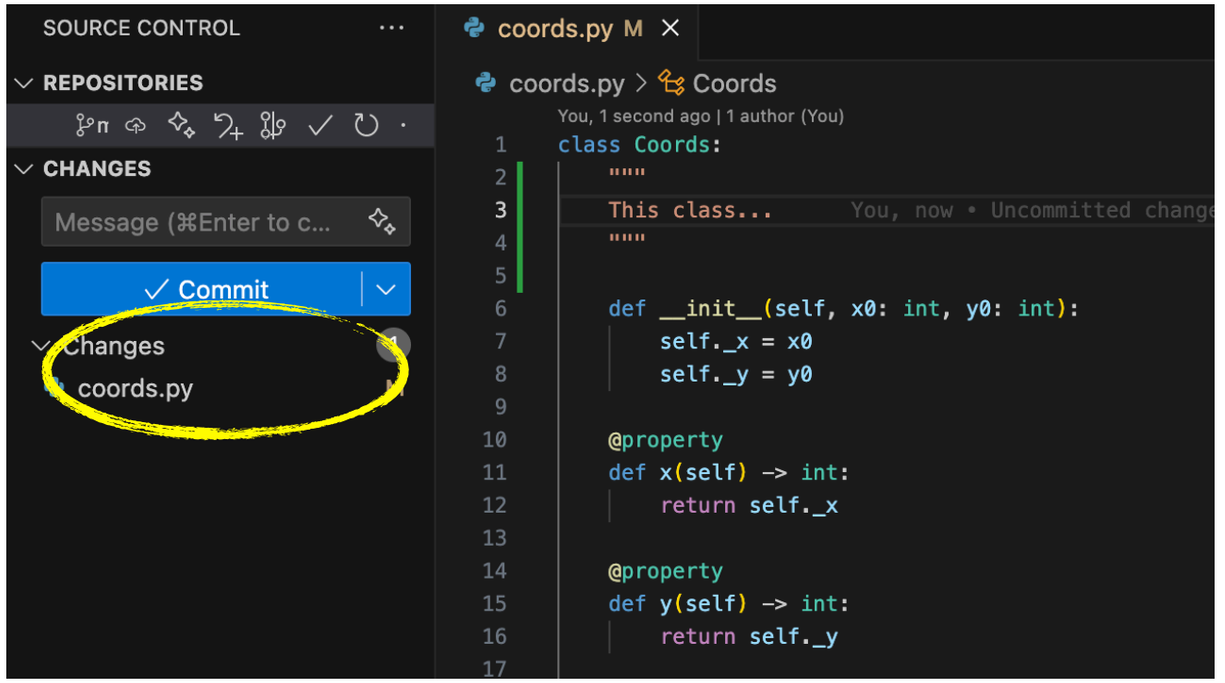
git status






```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what
will be committed)
  (use "git restore <file>..." to discard
changes in working directory)
    modified:   coords.py

no changes added to commit (use "git add"
and/or "git commit -a")
```



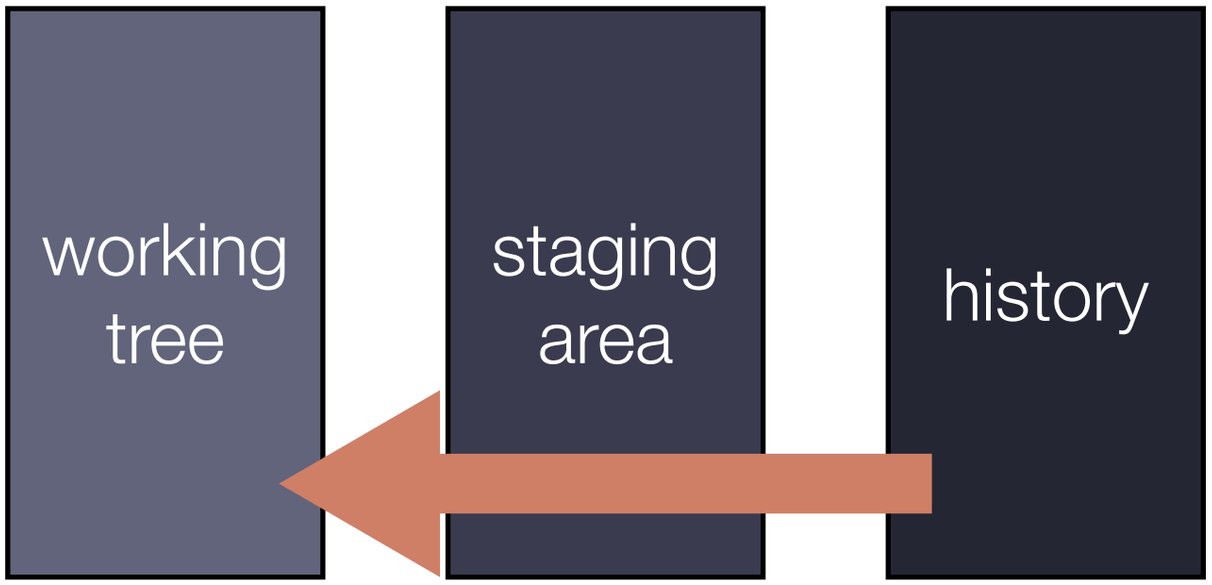
Récupération version enregistrée

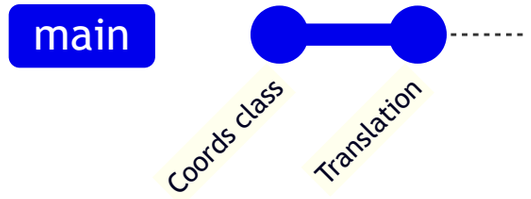
- `git restore <files>`

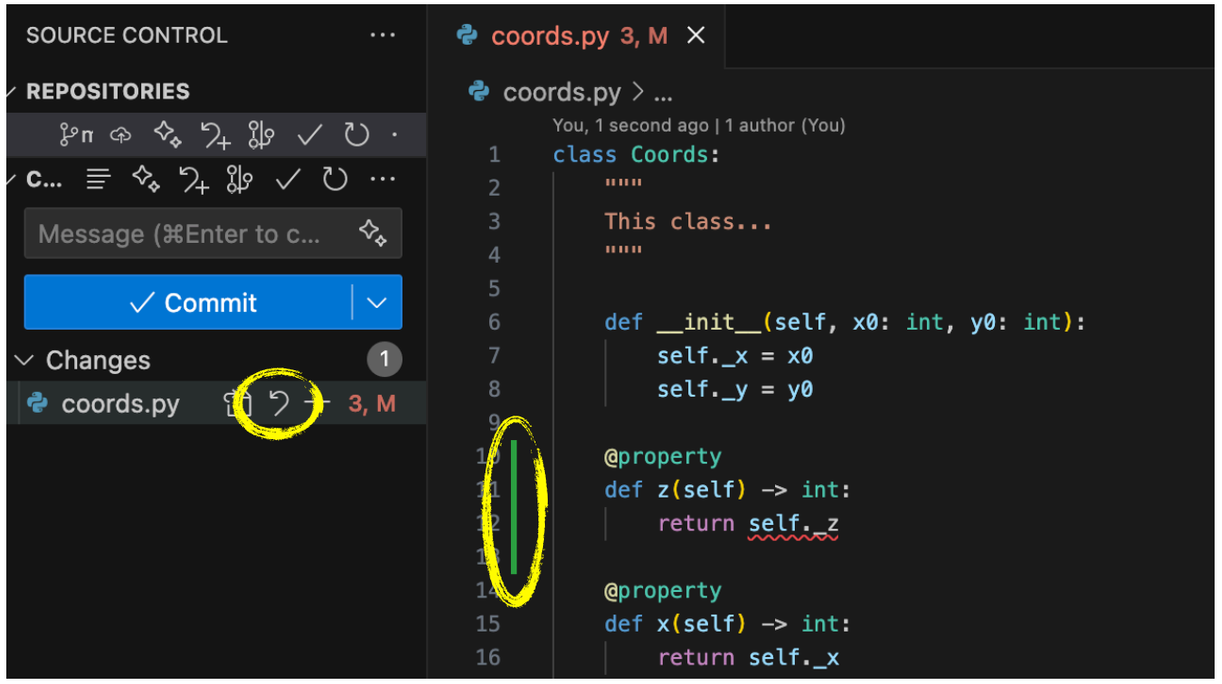
- Remplace les fichiers avec la version du dernier commit

- (Attention : pas d'annulation possible)

`git restore` *files*







Modifications préparées

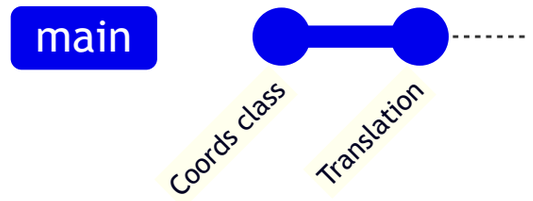
- `git diff --staged`

- Consultation pour le *prochain* commit

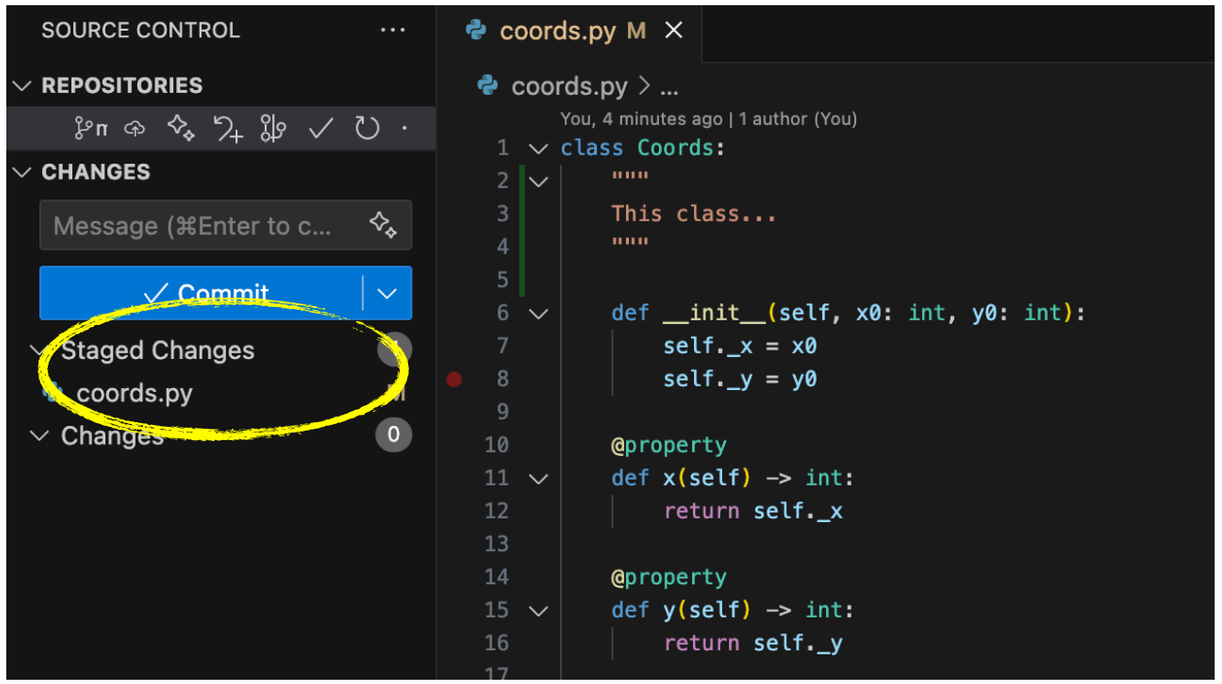
- (`git restore --staged <files>` pour enlever de l'index)

`git diff --staged`






```
--- a/coords.py
+++ b/coords.py
@@ -1,4 +1,8 @@
 class Coords:
+    """
+    This class...
+    """
+
     def __init__(self, x0: int, y0:
int):
         self._x = x0
```



Historique

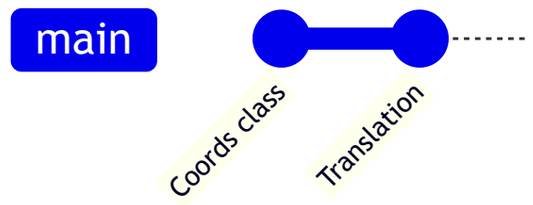
- `git log`

- Liste les commits avec identifiants, messages et métadonnées

- (`git show commit-id` pour voir le *diff*)

git log



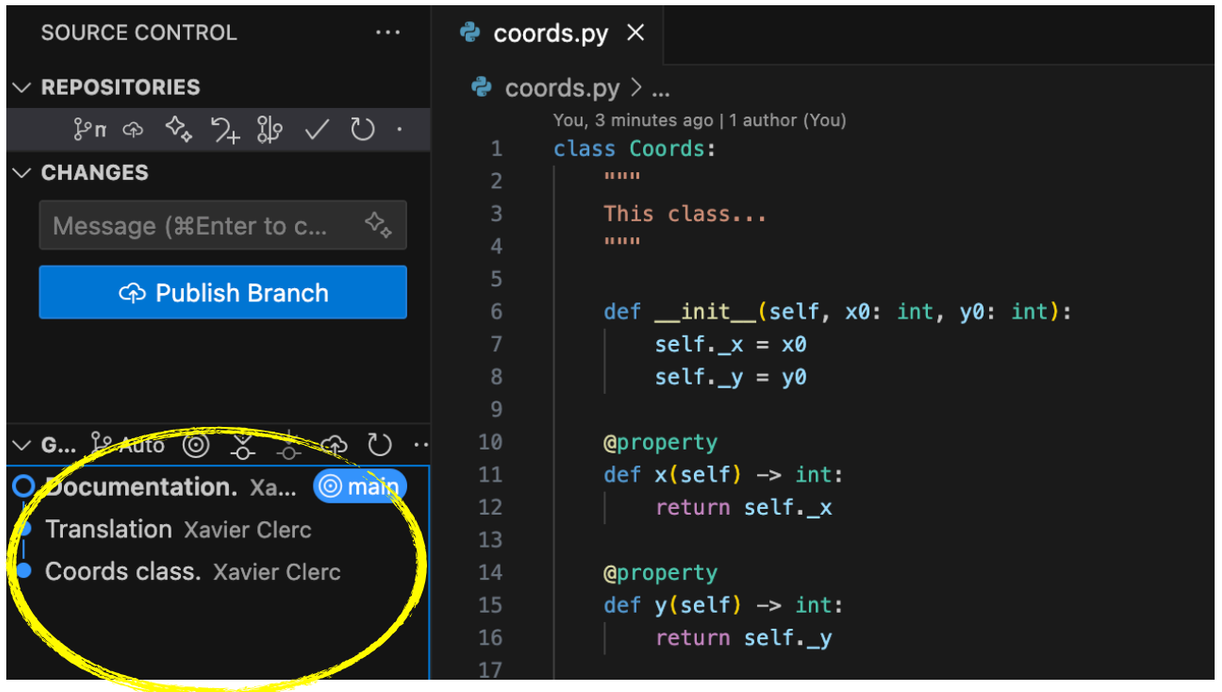



```
commit 816e57ecebe7108ce32476435fbb74
Author: xavier.clerc@enpc.fr
Date:   Mon Sep 1 ...

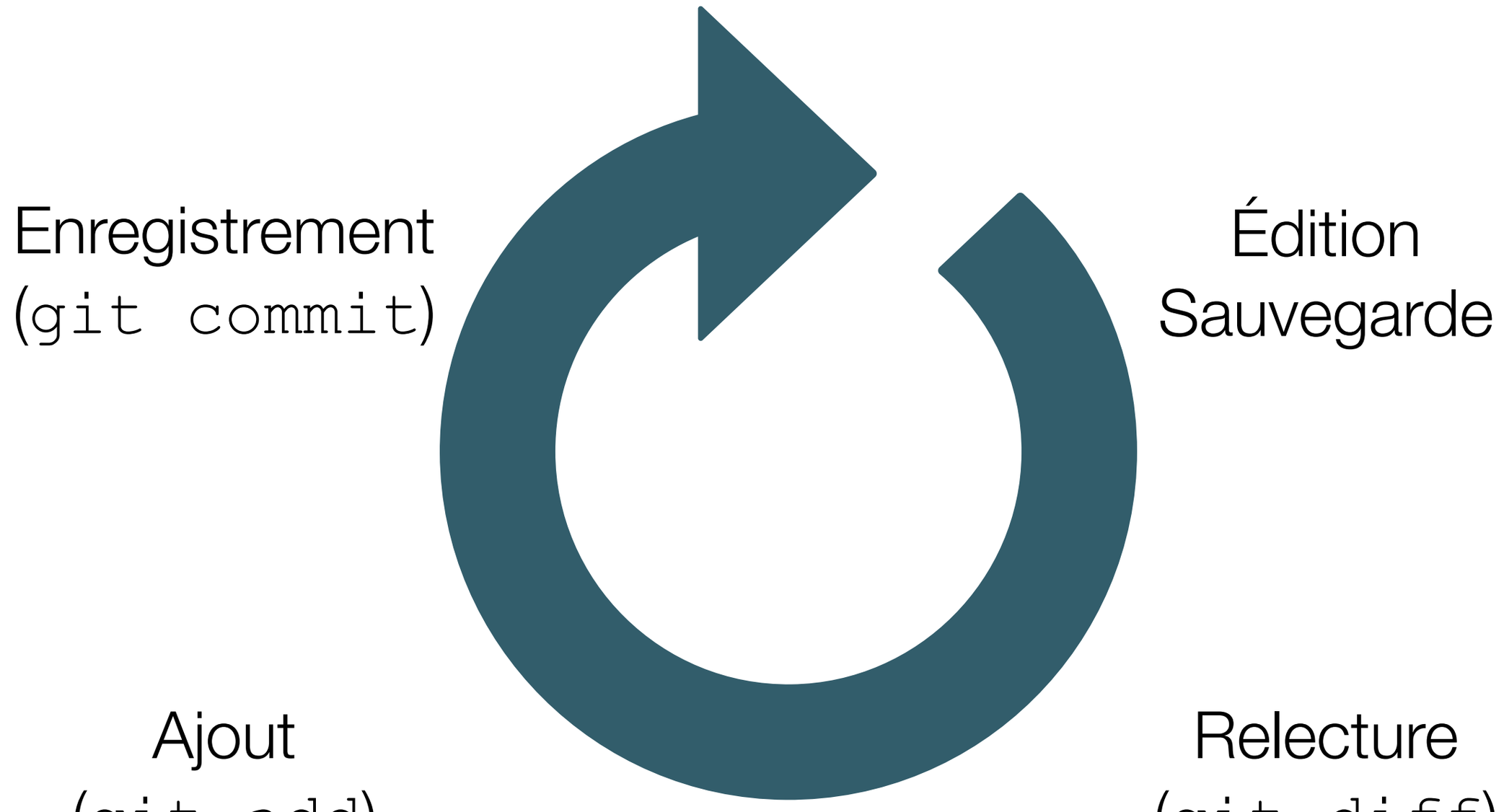
    Translation

commit fd45a73ab59add1ac3b6c497aa0038
Author: xavier.clerc@enpc.fr
Date:   Mon Sep 1 ...

    Coords class
```



Cycle de travail



Manipulations

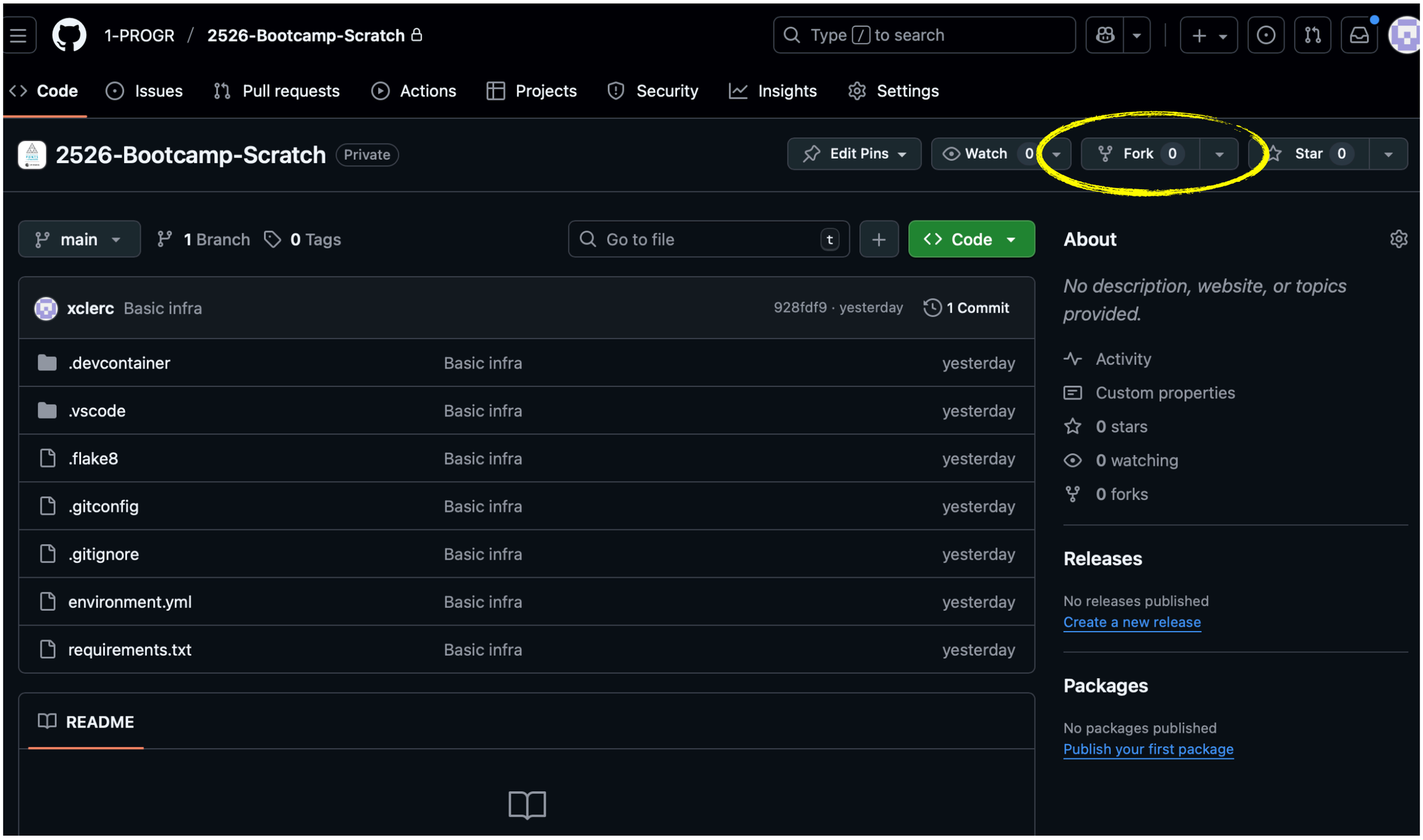
Manipulations

- VS Code dans navigateur

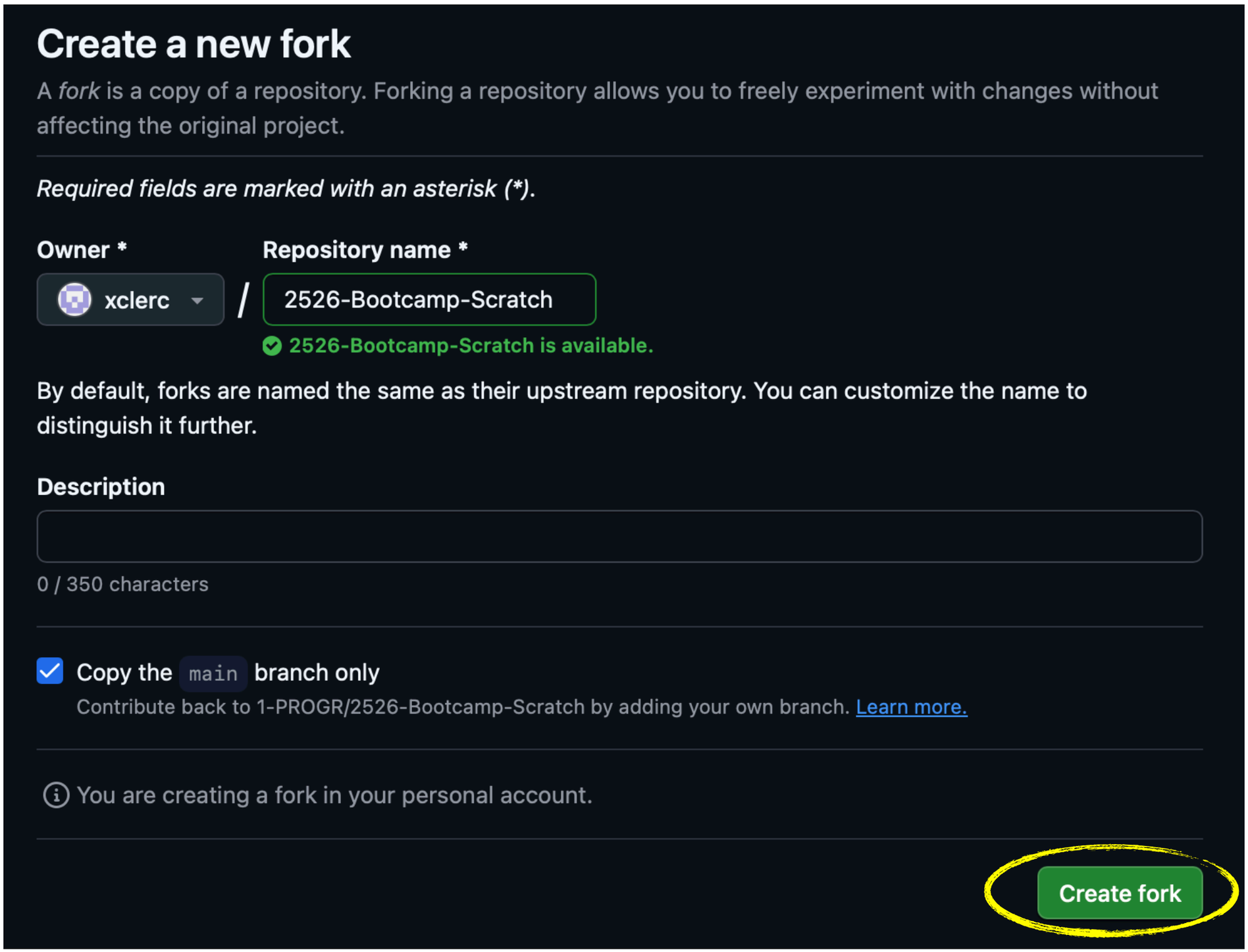
- (via GitHub Codespaces)

- Utilisation des commandes présentées

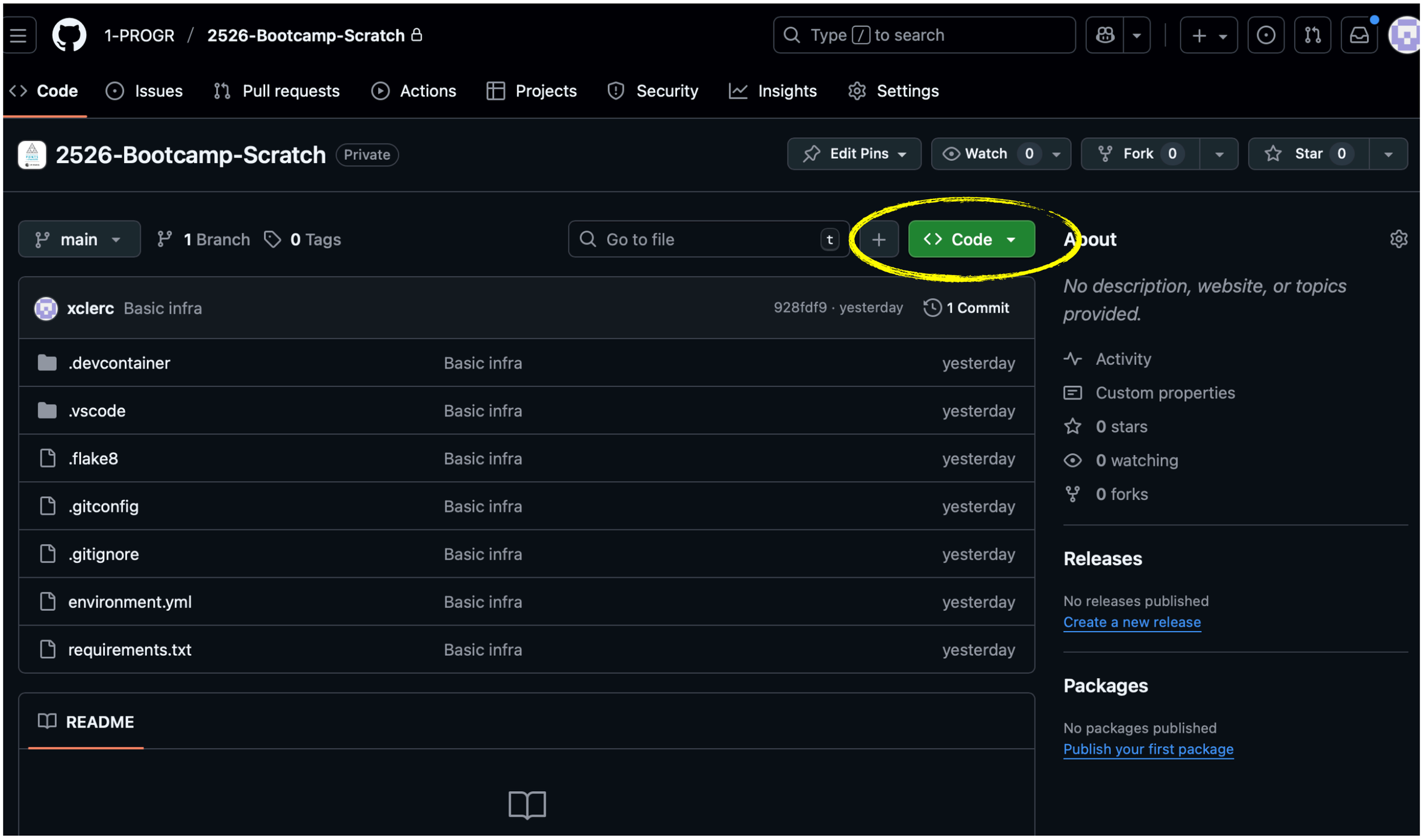
Lien sur Educnet



Création de votre copie



Lancement de VS Code en ligne



Lancement de VS Code en ligne

